AI AGENTIC PROJECTS

Complete Documentation

From Concept to Production

Search + Weather Agent · AI Travel Agent

LangChain · OpenAI GPT-4o-mini · Streamlit · ReAct Framework

Tavily Search · Weatherstack · ExchangeRate-API · RestCountries

A complete guide covering architecture, tools, code walkthrough,
live agent tracing, Streamlit UI design, and CLI deployment.

Table of Contents

1. Project Overview & Goals

This documentation covers two end-to-end AI Agent projects built using the **LangChain ReAct framework** with **OpenAI GPT-4o-mini**. Both projects demonstrate how to build intelligent agents that can reason, select tools, execute real-world API calls, and synthesise information into a coherent final answer.

What is an AI Agent?

An AI Agent is a system where a language model acts as a reasoning engine. Instead of just answering from memory, the agent can:

- Decide which external tool to use based on the user's query
- Call real APIs (weather, currency, search engines, etc.)
- Observe the result and decide what to do next
- Repeat the loop until it has enough information
- Generate a final, grounded answer based on real data

Two Projects Built

Project	Tools	Interface
Search + Weather Agent	Tavily Search, Weatherstack	Streamlit Web App + CLI
AI Travel Agent	Tavily Search, Weatherstack, ExchangeRate-API, RestCountries	Streamlit Web App + CLI

2. Tech Stack & Dependencies

Python Libraries

Tool / Component	Description
`langchain`	Core framework — agents, tools, prompts, callbacks
`langchain-openai`	LangChain integration for OpenAI / OpenRouter LLMs
`langchain-community`	Community tools including TavilySearchResults
`openai`	OpenAI SDK (used under the hood by langchain-openai)
`streamlit`	Web UI framework for both project frontends
`requests`	HTTP client for all external API calls
`python-dotenv`	Loads API keys from .env file
`certifi`	SSL certificate bundle — fixes SSL errors

External APIs

Tool / Component	Description
`OpenRouter / GPT-4o-mini`	LLM backbone — reasoning and final answer generation
`Tavily Search`	Real-time web search with structured JSON results
`Weatherstack`	Live weather data — temperature, humidity, wind
`ExchangeRate-API`	Live currency conversion rates
`RestCountries`	Free API — country info, flags, capitals, languages

Install All Dependencies

```
pip install langchain langchain-openai langchain-community
pip install openai streamlit requests python-dotenv certifi
```

3. Environment Setup (.env)

Create a file named **.env** in the root of your project. This file stores all your secret API keys. It is never committed to Git.

```
# .env file
OPENAI_API_KEY=your_openrouter_api_key_here
TAVILY_API_KEY=your_tavily_api_key_here
WEATHERSTACK_API_KEY=your_weatherstack_api_key_here
EXCHANGE_RATE_API_KEY=your_exchangerate_api_key_here
```

How Keys Are Loaded in Code

```
import os
from dotenv import load_dotenv
import certifi

os.environ['SSL_CERT_FILE'] = certifi.where()  # Fix SSL errors
load_dotenv()  # Read .env file

OPENAI_API_KEY       = os.getenv('OPENAI_API_KEY')
TAVILY_API_KEY       = os.getenv('TAVILY_API_KEY')
WEATHERSTACK_API_KEY = os.getenv('WEATHERSTACK_API_KEY')
EXCHANGE_RATE_API_KEY= os.getenv('EXCHANGE_RATE_API_KEY')
```

Why certifi? Python's SSL verification can fail on some systems when making HTTPS requests. Setting SSL_CERT_FILE to certifi's bundled CA certificates fixes this reliably.

Where to Get API Keys

Service	URL	Free Tier
OpenRouter	openrouter.ai	Yes — pay per token
Tavily	tavily.com	Yes — 1000 searches/month
Weatherstack	weatherstack.com	Yes — 250 calls/month
ExchangeRate-API	exchangerate-api.com	Yes — 1500 calls/month
RestCountries	restcountries.com	Yes — completely free

4. Project 1 — Search + Weather Agent

The first project is an agentic assistant capable of performing real-time **web searches** using Tavily and fetching **live weather data** using Weatherstack. It is exposed as a **Streamlit web app** with a modern dark UI and a live agent trace panel.

4.1 Architecture

Layer	Component	Purpose
Input	Streamlit text_input	User types their query
Reasoning	LangChain ReAct Agent	Decides which tools to call
LLM	GPT-4o-mini via OpenRouter	Generates thoughts & final answer
Tool 1	TavilySearchResults	Real-time web search (4 results)
Tool 2	get_weather_data	Live weather from Weatherstack API
Callback	StreamlitLogHandler	Streams agent trace to UI in real time
Output	Response card (Streamlit)	Displays final formatted answer

4.2 Tools Used

Tool 1 — TavilySearchResults

Tavily is a search API purpose-built for AI agents. Unlike Google scraping, it returns clean, structured JSON results that LLMs can easily reason over. Configured with `max_results=4` to return the top 4 web results.

```
from langchain_community.tools.tavily_search import TavilySearchResults
search_tool = TavilySearchResults(max_results=4)
```

Tool 2 — get_weather_data

A custom `@tool`-decorated function. The agent passes a city name, the function calls the Weatherstack API, and returns temperature, humidity, wind speed, and feels-like data as a formatted string.

```
@tool
def get_weather_data(city: str) -> str:
    url = f'https://api.weatherstack.com/current?'\
          f'access_key={WEATHERSTACK_API_KEY}&query;={city}'
    response = requests.get(url)
    data = response.json()
    return f'Temperature: {data["current"]["temperature"]}C'
```

4.3 Code Walkthrough

LLM Setup

ChatOpenAI pointed at OpenRouter's API with model openai/gpt-4o-mini, temperature=0 for deterministic outputs.

Prompt

hub.pull('hwchase17/react') loads the standard ReAct prompt from LangChain Hub — it instructs the LLM to think step-by-step and use tools.

create_react_agent

Combines LLM + tools + prompt into a ReAct agent object that knows how to reason and act.

AgentExecutor

Wraps the agent and manages the execution loop: call LLM → parse tool call → run tool → feed result back → repeat until Final Answer.

invoke()

Called with {input: user_query}. The executor runs the full loop and returns {output: final_answer}.

4.4 Streamlit UI — Modern Dark Intelligence Theme

The Streamlit app uses injected CSS to create a premium dark UI. Key design decisions:

- Syne (geometric display) + DM Mono (monospace) font pairing
- Deep #090b10 background with radial cyan glow at the top
- Capability pills showing: Web Search, Live Weather, ReAct Reasoning, AgentExecutor
- Suggestion chips as quick-start query examples
- Input box glows cyan on focus; button lifts on hover
- Response card animates in with a slideUp keyframe
- Custom scrollbar, hidden default Streamlit chrome

4.5 Live Agent Trace Panel

A custom class **StreamlitLogHandler** extends LangChain's `BaseCallbackHandler`. It is passed to the agent via `config={{"callbacks": [log_handler]}}`. Every internal step fires a callback method which appends an HTML row to a live Streamlit placeholder — creating a real-time execution trace.

Callback Method	Tag	What It Shows
`on_chain_start`	START	Query received by agent
`on_agent_action`	THINK + CALL	Tool decision + tool name & input
`on_tool_start`	FETCH	Tool is executing
`on_tool_end`	RESULT	First 220 chars of tool output
`on_tool_error`	ERROR	Any tool failure message
`on_agent_finish`	DONE	Agent finished, composing answer

5. Project 2 — AI Travel Agent

The second project extends the agent with two additional tools — **currency conversion** and **country information** — making it a full-featured travel intelligence assistant. It also ships with a **CLI version (main.py)** that runs entirely in the terminal.

5.1 Architecture

Same ReAct loop as Project 1, with 4 tools instead of 2:

- TavilySearchResults — web search for tourist attractions, visa info, etc.
- get_weather_data — live weather for any city
- get_exchange_rate — real-time currency conversion via ExchangeRate-API
- get_country_info — capital, currency, population, languages, flag via RestCountries

5.2 All 4 Tools Explained

Tool 1 — get_country_info

Calls the free RestCountries API with a country name. Returns a formatted string with: common name, capital, region, population (formatted with commas), currency name and code, languages, ISO country codes (2-letter and 3-letter), and a URL to the country's flag PNG.

```
url = f'https://restcountries.com/v3.1/name/{country}'
     '?fields=name,capital,currencies,population,region,languages,flags,cca2,cca3'
```

Tool 2 — get_exchange_rate

Accepts a comma-separated currency pair string like `INR,JPY` or `USD,EUR`. Calls the ExchangeRate-API v6 endpoint to get latest rates. Returns a single line: '1 INR = 1.68 JPY'.

```
url = f'https://v6.exchangerate-api.com/v6/{EXCHANGE_RATE_API_KEY}/latest/{from_currency}'
rate = data['conversion_rates'][to_currency]
```

Tool 3 — get_weather_data

Same as Project 1 but additionally returns wind speed and feels-like temperature.

Tool 4 — TavilySearchResults

Same as Project 1 — used for tourist attractions, visa requirements, travel tips.

5.3 Code Walkthrough

The agent setup follows the exact same pattern as Project 1:

```
tools = [search_tool, get_weather_data, get_exchange_rate, get_country_info]

agent = create_react_agent(llm=llm, tools=tools, prompt=prompt)

agent_executor = AgentExecutor(
    agent=agent,
    tools=tools,
    verbose=True,
    handle_parsing_errors=True
)
```

handle_parsing_errors=True — If the LLM generates a malformed tool call (wrong argument format, missing fields), the executor catches the error and asks the LLM to try again instead of crashing. Essential for production robustness.

5.4 Streamlit UI — Luxury Travel Editorial Theme

The travel agent UI uses a completely different aesthetic from Project 1:

- Playfair Display serif (italic gold) + Outfit sans + JetBrains Mono — editorial feel
- Deep ink background with warm gold radial glow (travel luxury vibe)
- Decorative gold horizontal rules with gradient fade
- 4-tool badge strip at the top: AgentExecutor, Country Info, Exchange Rates, Live Weather, Tavily
- 2x2 sample query grid with destination emoji cards
- text_area instead of text_input — allows multi-line complex travel queries
- macOS-style traffic light dots on the trace panel header with live step counter
- Response card with Playfair Display header, gold top border, and model metadata line

5.5 CLI Version — main.py

The CLI version strips away all Streamlit code and runs in a simple terminal loop. Perfect for testing, scripting, or when you don't need a UI.

```
# Run it
python main.py

# Output
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■       ✈   AI Travel Agent  (CLI Mode)   ✈       ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
You: I am travelling to Japan...
```

Key CLI features:

- Interactive while-loop — keeps running, ask multiple queries without restarting
- verbose=True on AgentExecutor prints the full reasoning trace to terminal automatically
- Clean FINAL ANSWER block separated by dashed dividers after the trace
- Type 'exit', 'quit', or Ctrl+C to stop gracefully
- Empty input is silently skipped — no crash

6. LangChain ReAct Framework — Deep Dive

ReAct stands for **Reasoning + Acting**. It is a prompting framework that interleaves chain-of-thought reasoning with tool use. Published in the paper 'ReAct: Synergizing Reasoning and Acting in Language Models' (2022).

The ReAct Loop

1. Thought

The LLM reasons about what it needs to do next. Example: 'I need to find the capital of Japan first.'

2. Action

The LLM outputs a tool name and its input. Example: Action: get_country_info, Action Input: Japan

3. Observation

The tool runs and returns its result. This is fed back into the LLM's context.

4. Repeat

The LLM reads the observation and decides: do I need another tool, or do I have enough to answer?

5. Final Answer

When the LLM has enough information, it outputs 'Final Answer:' and the executor returns that to the user.

The ReAct Prompt

Both projects use `hub.pull('hwchase17/react')`, the canonical ReAct prompt from LangChain Hub. It provides the LLM with:

- Instructions on the thought-action-observation format
- A list of available tools and their descriptions
- The scratchpad (history of all previous thoughts, actions, observations)
- The current user input

The tool descriptions (the docstrings on your @tool functions) are critical — the LLM reads them to decide which tool is appropriate for each step.

7. AgentExecutor Explained

AgentExecutor is the engine that runs the agent. It manages the loop, handles tool execution, and collects the final answer.

```
agent_executor = AgentExecutor(
    agent=agent,              # The ReAct agent object
    tools=tools,              # List of tool objects
    verbose=True,             # Print trace to stdout
    handle_parsing_errors=True # Retry on malformed LLM output
)
```

Key Parameters

Parameter	Type	Purpose
`agent`	`BaseAgent`	The ReAct agent with LLM + prompt
`tools`	`List[BaseTool]`	All tools the agent can call
`verbose`	`bool`	Prints full reasoning trace to stdout
`handle_parsing_errors`	`bool`	Retries if LLM output is malformed
`max_iterations`	`int (default 15)`	Stops runaway loops after N steps
`max_execution_time`	`float`	Timeout in seconds for the full run

8. Live Logging with BaseCallbackHandler

LangChain's callback system lets you hook into every step of agent execution. Both Streamlit apps define a custom handler that inherits from `BaseCallbackHandler` and renders a live HTML trace panel.

```
from langchain.callbacks.base import BaseCallbackHandler

class TravelLogHandler(BaseCallbackHandler):
    def __init__(self, placeholder):  # st.empty() placeholder
        self.ph = placeholder
        self.rows = []

    def on_agent_action(self, action, **kwargs):
        self.rows.append(render_row('THINK', action.tool))
        self.ph.markdown(build_panel(self.rows), unsafe_allow_html=True)

    def on_tool_end(self, output, **kwargs):
        self.rows.append(render_row('RESULT', output[:220]))
        self.ph.markdown(build_panel(self.rows), unsafe_allow_html=True)
```

How to Wire the Handler

```
handler = TravelLogHandler(log_placeholder)

response = agent_executor.invoke(
    {'input': user_query},
    config={'callbacks': [handler]}   # <-- Pass handler here
)
```

The handler is passed per-invocation via the config dict. This means each user query gets its own fresh handler with an empty log, which is the correct pattern for Streamlit apps.

9. Sample Queries & Expected Output

Best Query to Test All 4 Tools

```
I am travelling from India to Japan.
Tell me about Japan including its capital, currency, population and languages.
Convert 10000 INR to Japanese Yen.
Tell me the current weather in Tokyo.
Also suggest the top 5 tourist attractions in Tokyo.
```

Agent will use tools in this order:

- get_country_info(Japan) → capital, currency, population, languages, flag URL
- get_exchange_rate(INR,JPY) → 1 INR = 1.68 JPY, so 10000 INR = 16800 JPY
- get_weather_data(Tokyo) → temperature, weather description, humidity, wind
- tavily_search_results_json(top tourist attractions in Tokyo) → structured web results

Other Useful Queries

Query Type	Example
France Trip	Planning a trip to France. Share country info, convert 5000 INR to EUR, weather in Paris and must-see spots.
Dubai Trip	I am going to Dubai. Give me UAE country info, convert 10000 INR to AED, check Dubai weather and top tourist p
USA Trip	Trip to New York. Tell me about USA, convert 20000 INR to USD, current weather in NYC and best attractions.
Weather only	What is the current weather in London, Tokyo and Sydney?
Currency only	Convert 50000 INR to Japanese Yen, Euro, and US Dollar.
Search only	What are the visa requirements for Indians travelling to Japan in 2025?

10. How to Run Both Projects

File Structure

```
project/
■■■ .env                    # API keys
■■■ app.py                  # Project 1 — Search + Weather (Streamlit)
■■■ travel_agent_app.py     # Project 2 — Travel Agent (Streamlit)
■■■ main.py                 # Project 2 — Travel Agent (CLI)
```

Run Project 1 — Search + Weather Agent

```
streamlit run app.py
```

Run Project 2 — Travel Agent (Streamlit)

```
streamlit run travel_agent_app.py
```

Run Project 2 — Travel Agent (CLI)

```
python main.py
```

First Time Setup

```
# 1. Clone or create your project folder
mkdir ai-agents && cd ai-agents

# 2. Create virtual environment
python -m venv venv
source venv/bin/activate   # Windows: venv\Scripts\activate

# 3. Install dependencies
pip install langchain langchain-openai langchain-community
pip install openai streamlit requests python-dotenv certifi

# 4. Create .env with your API keys
# 5. Run!
streamlit run travel_agent_app.py
```

11. Troubleshooting & Tips

SSL Certificate Error

Add os.environ['SSL_CERT_FILE'] = certifi.where() before any requests. This is already in all project files.

API key not found (None)

Make sure your .env file is in the same directory you run the script from. Check for typos in key names.

Agent stuck in a loop

Add max_iterations=10 to AgentExecutor. The default is 15 which is usually fine but some queries can loop.

Tavily returns no results

Tavily free tier has a monthly limit. Check your usage at app.tavily.com. Also try a more specific query.

Weatherstack returns error

Free tier only supports HTTP (not HTTPS). Check your access_key is correct and not expired.

handle_parsing_errors not helping

If the LLM keeps generating wrong output, try switching to a stronger model like gpt-4o instead of gpt-4o-mini.

Streamlit reruns on every interaction

This is normal Streamlit behaviour. The agent and executor are recreated each time. For production, use st.cache_resource on the executor.

Exchange rate tool fails

Make sure currency codes are valid ISO 4217 codes (INR, JPY, USD, EUR, AED, GBP, etc.). The tool auto-uppercases them.

End of Documentation
AI Agentic Projects — LangChain · OpenAI · Streamlit